Cairo University

Faculty of Engineering

Electronics and Electrical Communications Engineering Department

Third Year

Digital Communications Project II

Name	Section	ID
Serag Khaled Ahmed Abdelaal	2	9221268
Sama Saeed Mohamed Mohamed	2	9220371
Sohaila Ahmed Mohamed Abdelmageed	2	9220377
Manar Kamal Morsi Ahmed	4	9220869
Tarek Osama Mahmoud Eletriby	2	9220414

Role of each member

- **Sama & Serag: Requirement 1**
- **Sohaila & Manar: Requirement 2**
- **Tarek: Requirement 3**

Requirement 1:

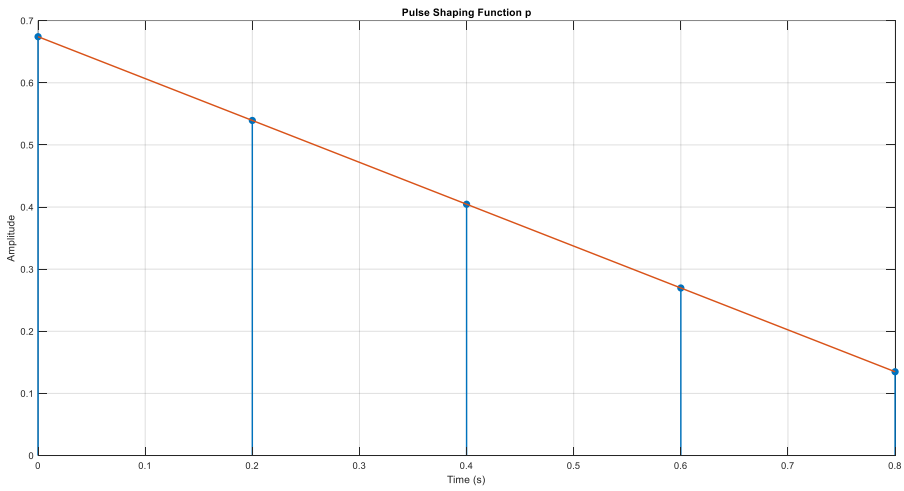


Figure 1: Pulse shaping function

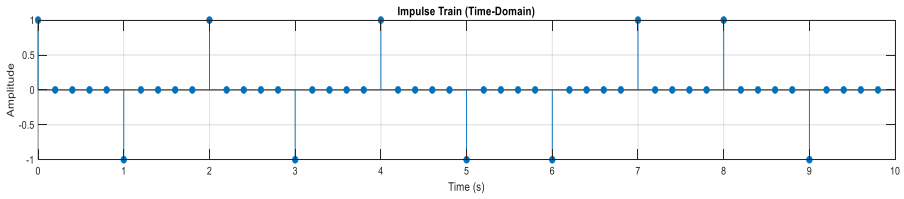


Figure 2: Impulse Train

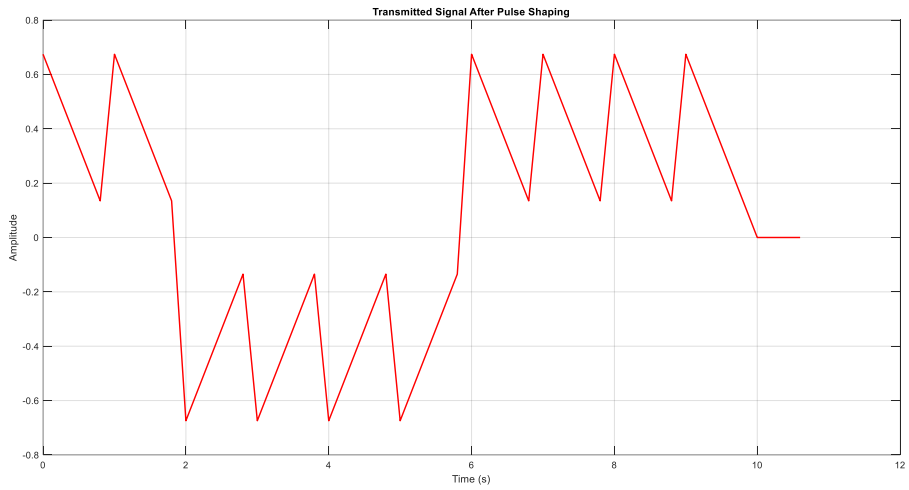


Figure 3: Transmitted Signal After pulse shaping

Comment :

- The pulse shaping function shown in **Figure 1** defines how each bit is represented in time. This specific pulse, normalized to unit energy, gradually decreases in amplitude from 5 to 1 over the symbol period Ts=1 sec, sampled every 200 ms to match the system's discrete-time implementation.
- In **Figure 2**, the impulse train represents the upsampled version of the bits, where each bit is followed by four zeros due to the upsampling factor of 5.
- The transmitted signal after pulse shaping, shown in **Figure 3**, represents the result of convolving the impulse train with the pulse shaping function.

I. Part A

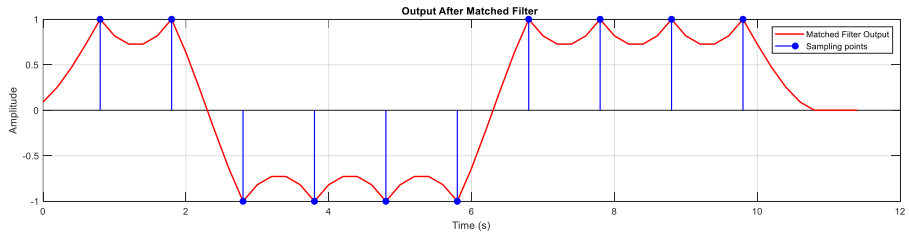


Figure 4: Output After Matched Filter

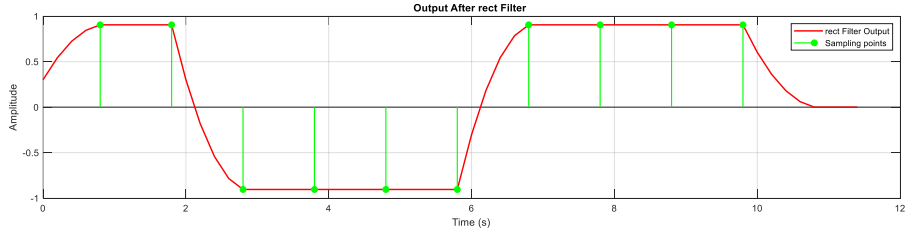


Figure 5: Output After rect Filter

Comment:

- **Figure 4** shows the output of the **matched filter**, which is designed to mirror the pulse shape used in transmission. The filtered signal exhibits smooth peaks at locations corresponding to symbol intervals. The **blue markers** represent the sampling points, which align with the peaks of the filter output, allowing for accurate detection of transmitted symbols.
- **Figure 5** shows the output of the **rect filter,which is the non-matched filter.**The **green vertical lines** represent the **sampling points**, which are taken at the correct symbol timing instants. Although the samples still reflect the transmitted symbols, the overall waveform Amplitude decreased (**as expected**), especially in regions where symbol transitions occur. This makes the rectangular filter less optimal compared to the matched filter.

II. Part B

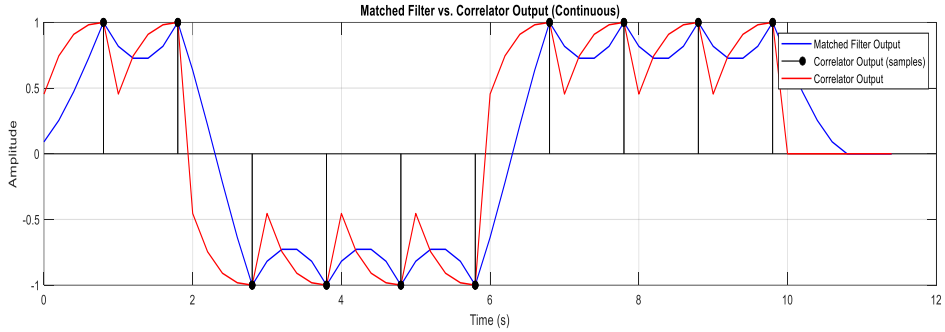


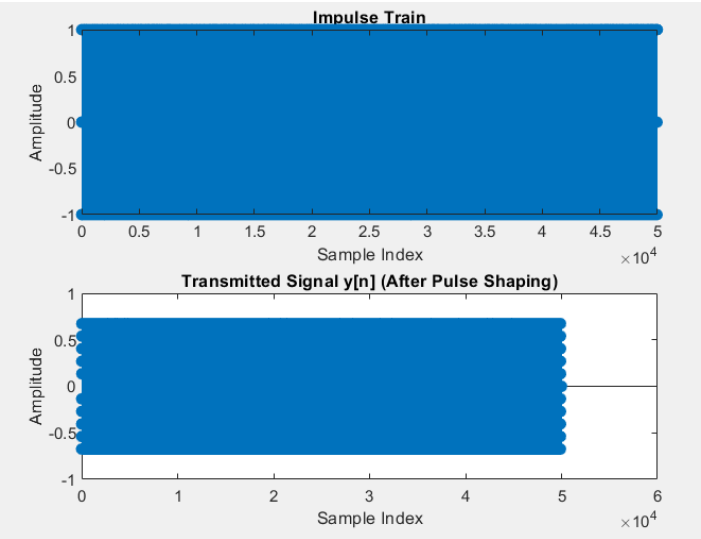
Figure 6: Matched Filter output VS. Correlator output

Comment:

- The matched filter output (**blue curve**) shows the response due to the convolution with the time-reversed pulse shape. The correlator output (**red curve**), which represents the inner product of the received signal with the expected pulse shape, is also shown. Both outputs peak around the sampling points (**black samples**), allowing for correct symbol detection. Although the shapes differ slightly, both methods have the same detection performance.

Requirement 2:

a) Signal Size = 10000



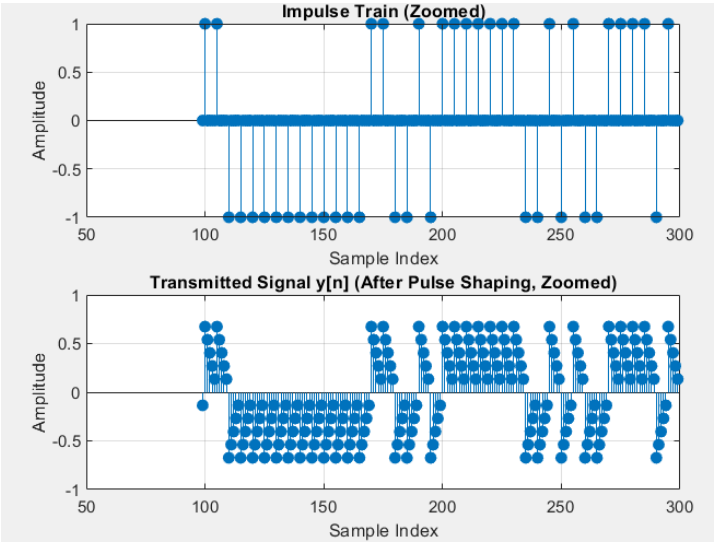


Figure 7: Impulse train and Y[n] (transmitted signal)

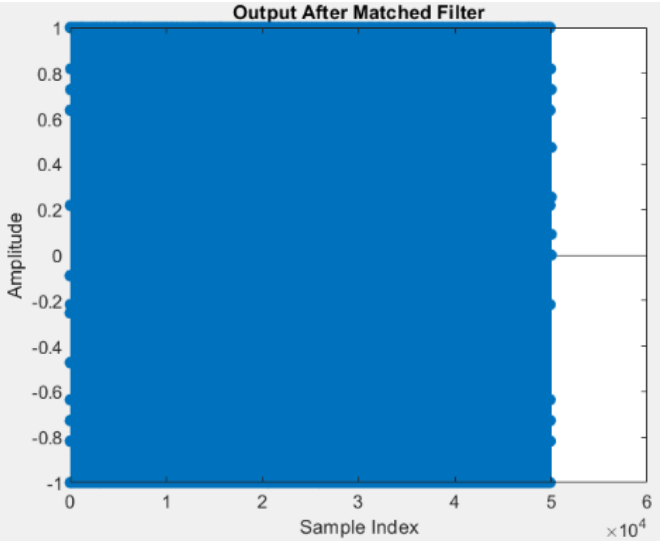
Comments:

Impulse Train:

- This plot shows the upsampled PAM signal.
- The values alternate between +1 and -1, corresponding to logic '1' and '0' in Polar NRZ format.

Transmitted Signal y[n]:

- This is the transmitted signal after pulse shaping, achieved by convolving the impulse train with the pulse shape p[n].
- The result is a smooth waveform, with reduced bandwidth and minimized intersymbol interference (ISI).
- This waveform is what gets sent over the channel, and it's more suitable for transmission than the original impulse train.



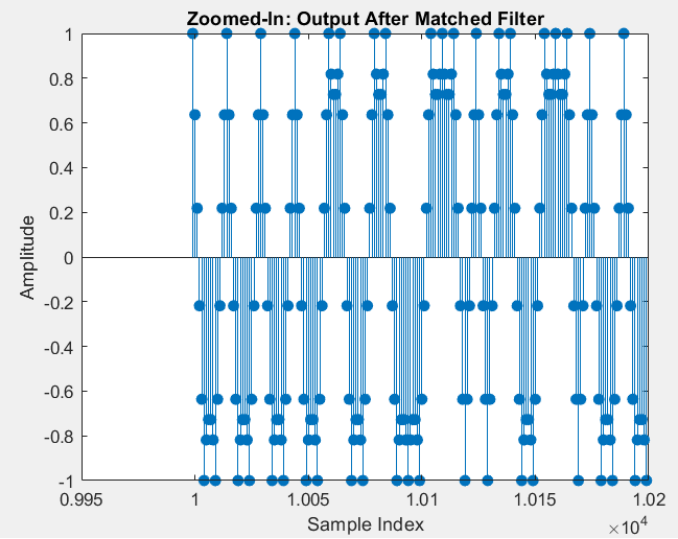
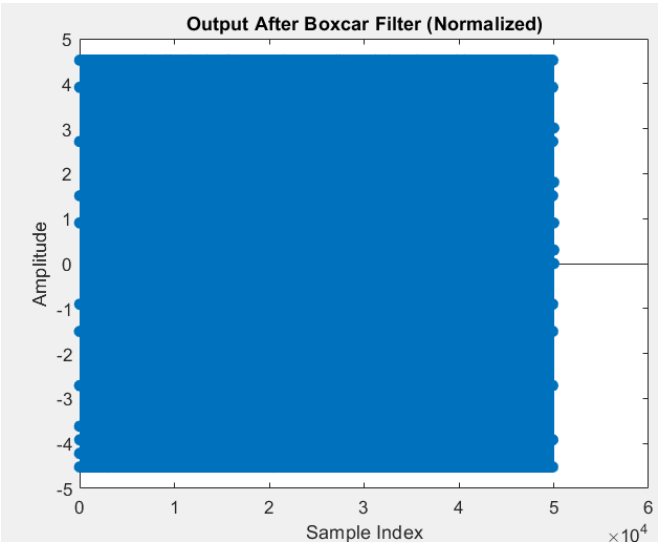


Figure 8: Output of the matched filter

Comments:

Matched Filter Output:

- This shows the signal after applying the matched filter, which is the time-reversed version of the transmit pulse. The purpose of the matched filter is to maximize the SNR at the sampling instant.
- The matched filter successfully emphasizes symbol energy at the right sampling instances, enabling accurate detection.



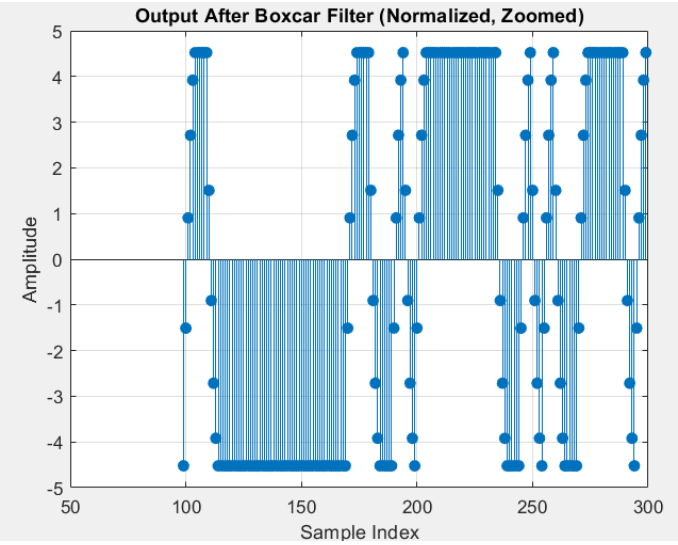


Figure 9: Output of the Boxcar (non-matched filter)

Comments:

Non-matched Filter Output:

- The effect of the rectangular pulse (boxcar) filter is seen as it averages over a rectangular window.
- The repeated rectangular shape indicates smoothing, but it may not be the best for minimizing noise (as explored in the BER analysis).
- This filter does not match the transmitted pulse, which leads to a poor performance in the presence of noise.

b) Noise Analysis

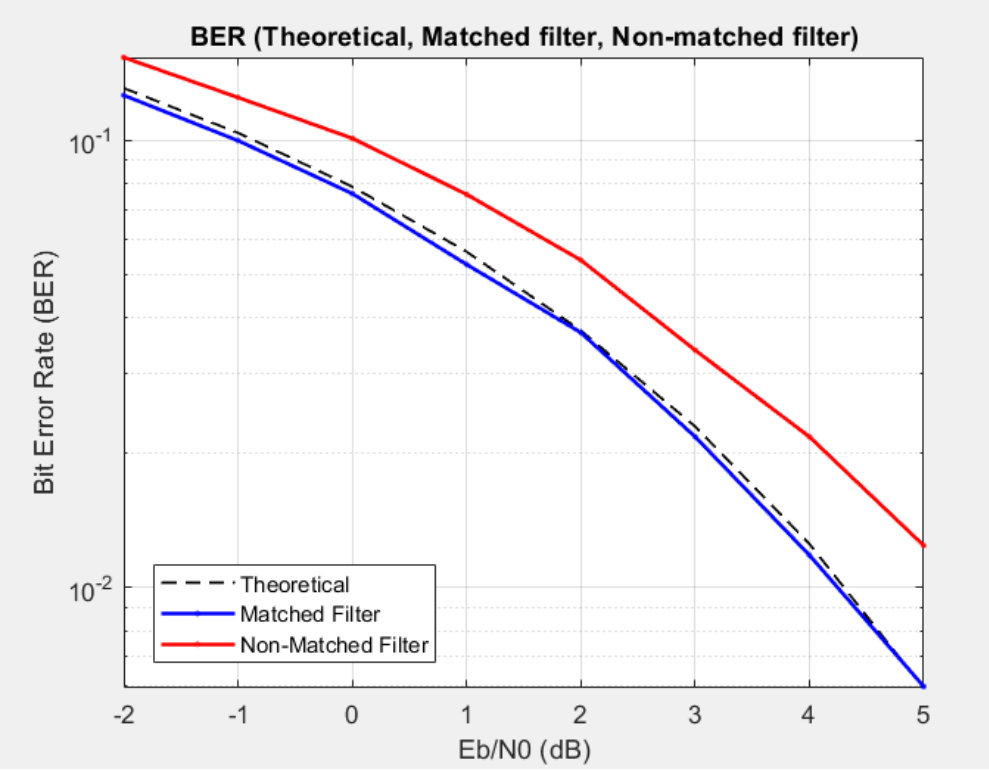


Figure 10: BER for Matched and Non-matched filter Vs Theoretical

Comments:

The BER vs. Eb/N0 plot demonstrates the good performance advantage of using a matched filter over a non-matched (boxcar) filter in a PAM communication system. While both filters show improvement in BER with increasing Eb/N0, the matched filter closely follows the theoretical performance curve, confirming it's the best in minimizing

bit errors in the presence of noise. In contrast, the boxcar filter has higher BER across all SNR levels due to its mismatch with the transmitted pulse shape, showing the importance of filter design in achieving reliable communication.

So we can conclude that:

- The matched filter significantly reduces BER compared to a non-matched boxcar filter.
- Proper filter design is essential in digital communication systems, especially in noisy environments.
- Even a simple mismatch (like using a rectangular filter instead of a pulse-matched one) can lead to noticeable degradation.

Requirement 3:

Case a:

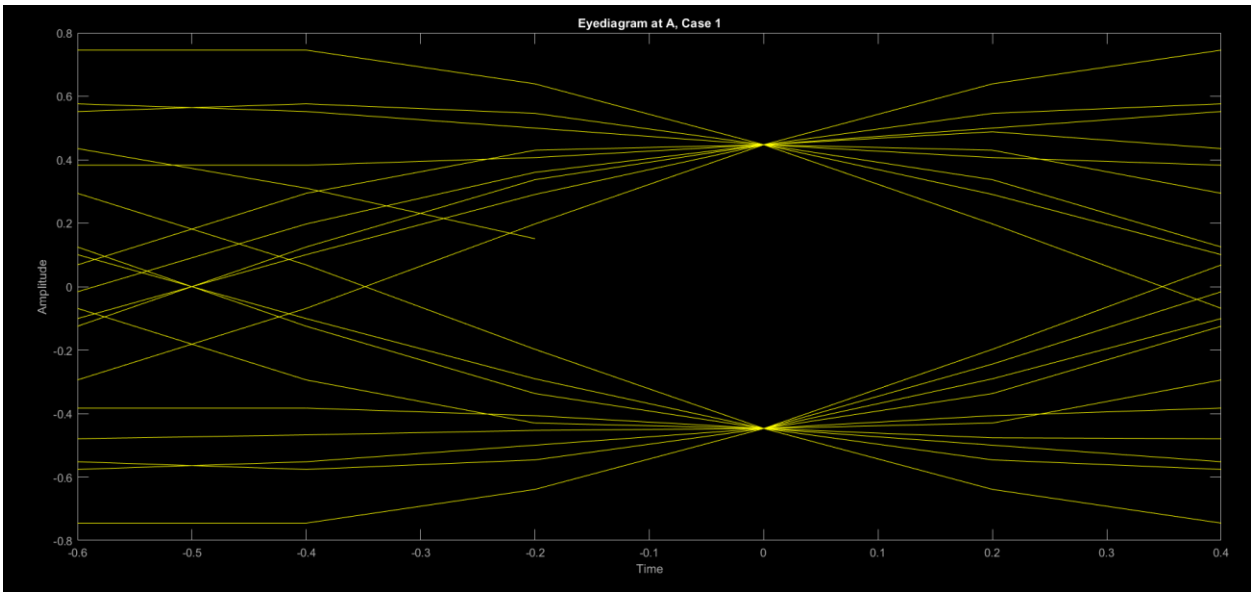


Figure 11 Eyediagram for case a at point A

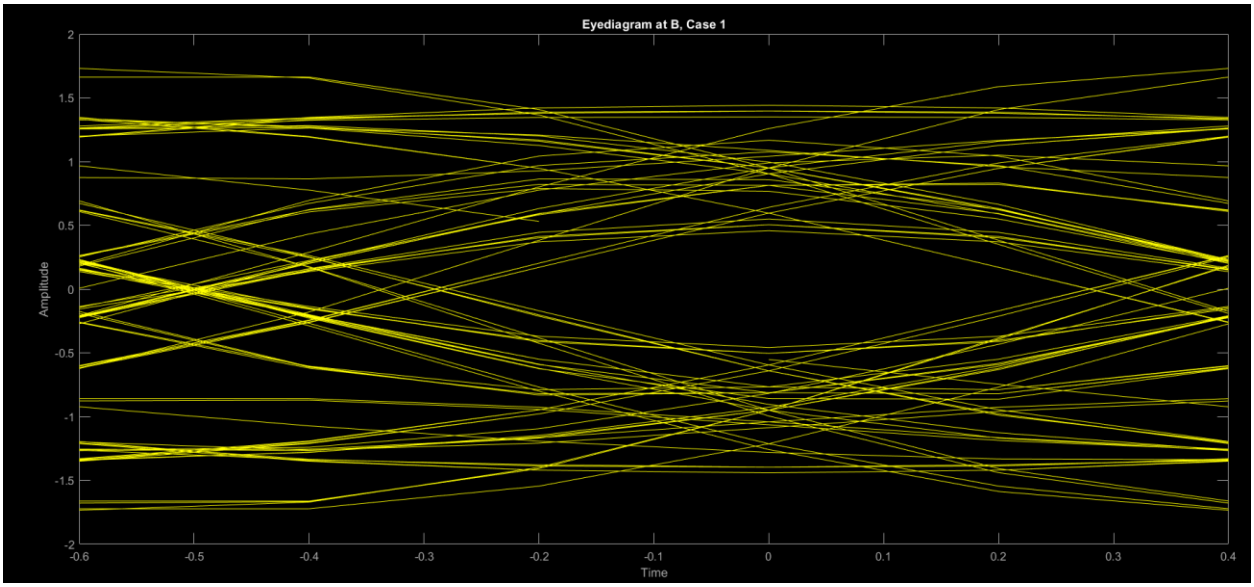


Figure 12 Eyediagram for case a at point B

Case b:

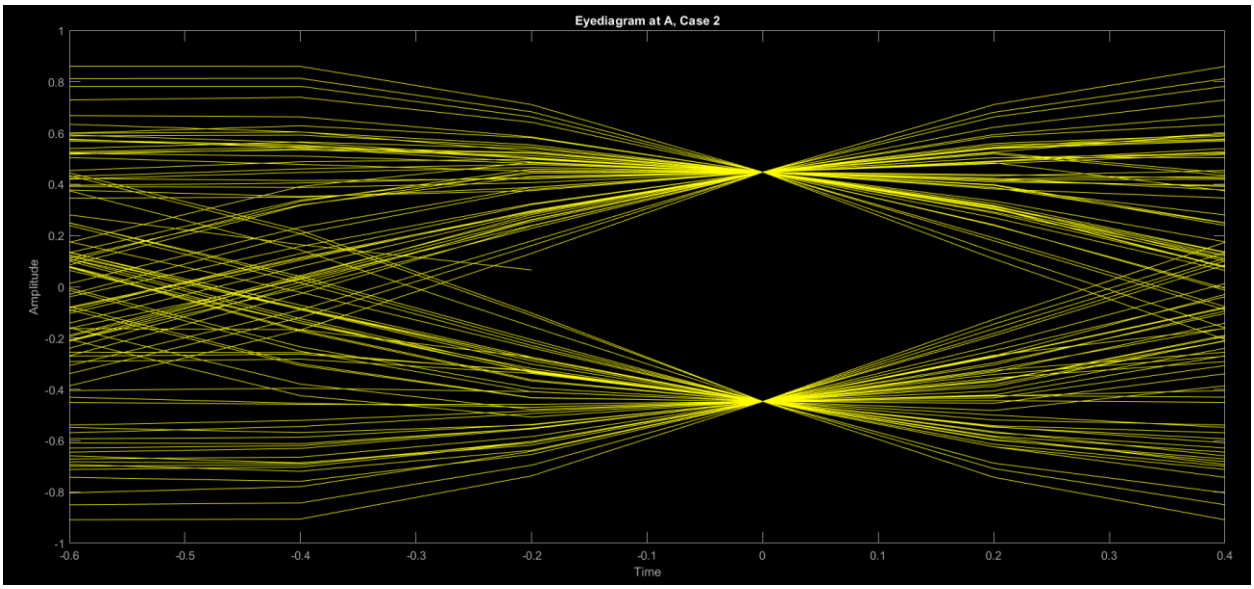


Figure 13 Eyediagram for case b at point A

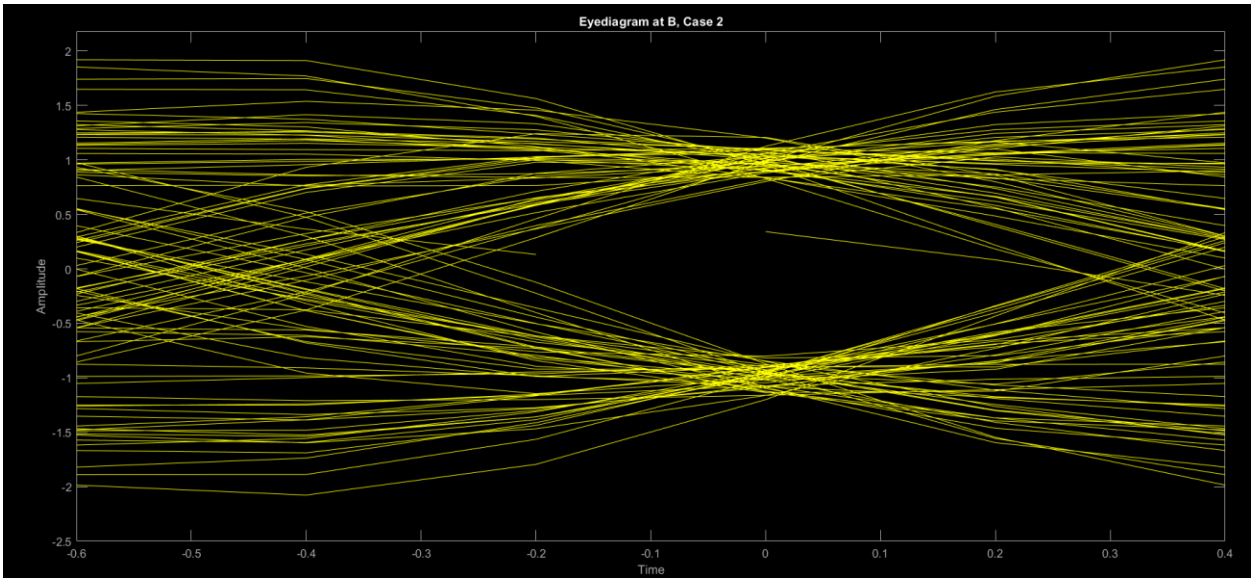


Figure 14 Eyediagram for case b at point B

Case c:

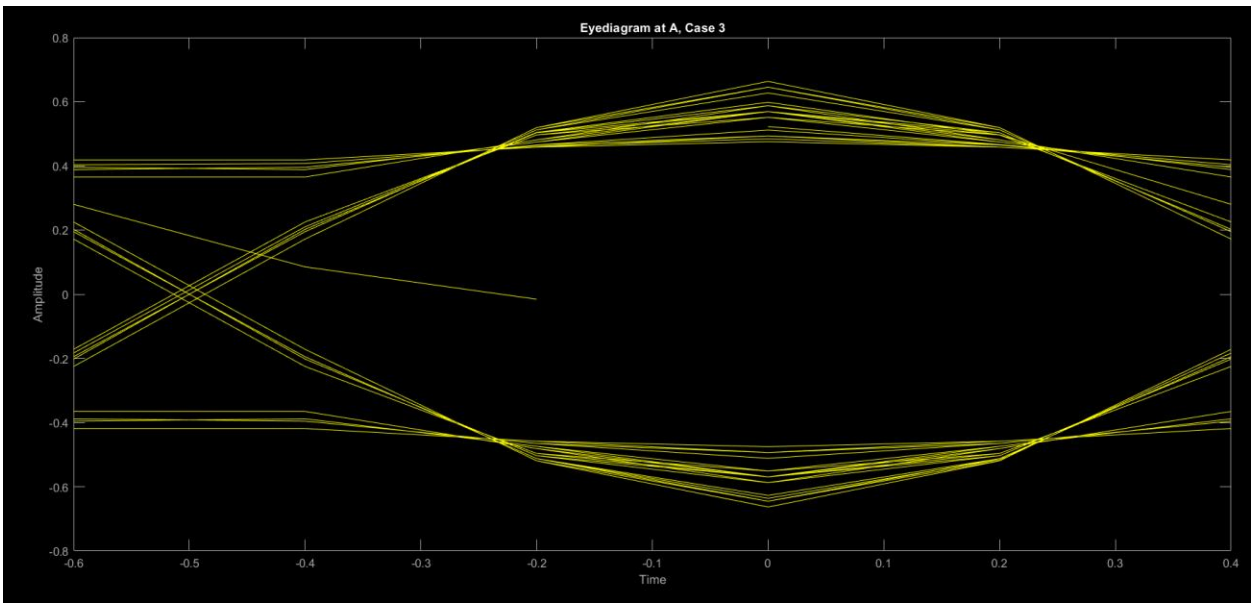


Figure 15 Eyediagram for case c at point A

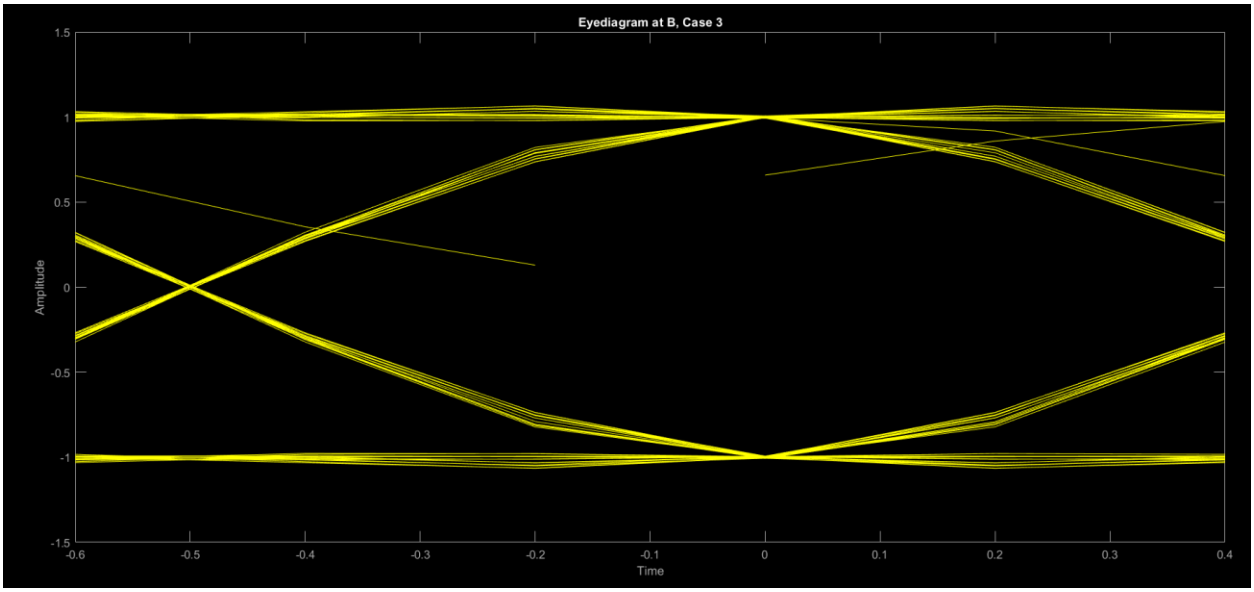


Figure 16 Eyediagram for case c at point B

Case d:

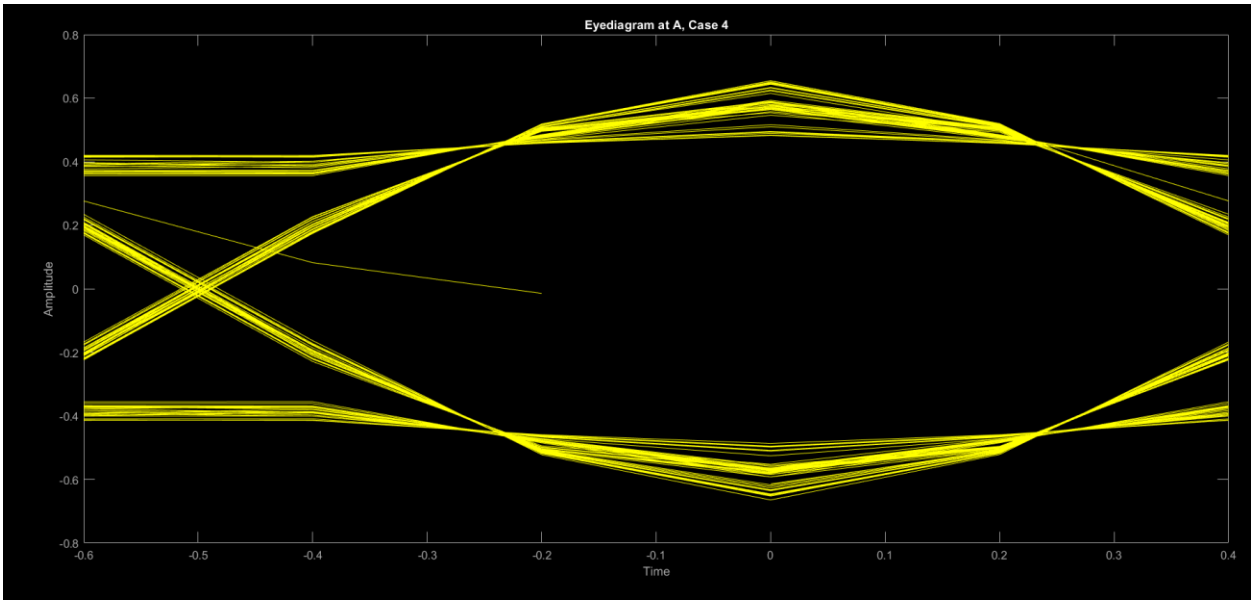


Figure 17 Eyediagram for case d at point A

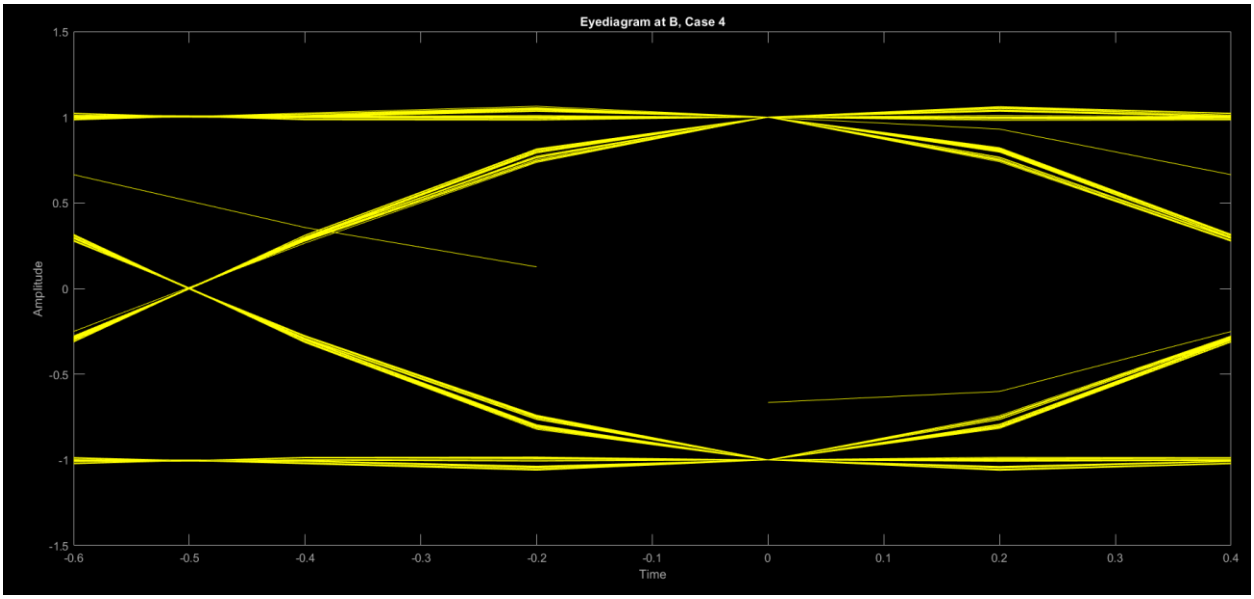


Figure 18 Eyediagram for case d at point B

Comment on the resulting eyediagrams:

- The higher the delay, the lower the distortion, as the filter better approximates the ideal infinite impulse response.
- The higher the roll-off factor the less ISI occurs. The best roll-off factor in terms of minimizing ISI is R= 1, but it comes at the cost of the increased bandwidth.
- At R=0 the highest ISI occurs, but the smallest bandwidth is achieved.
- The best eyediagram (with the lowest ISI) is observed when R=1 and delay=8.
- The most suitable sampling instant is at the point of widest eye opening, which is clearly observed at the middle of the pulse.

Appendix:

```
%% Parameters
A = 1;
Ts = 1;                     % Sampling time in sec
SignalSize = 10;            %Size of random signal
p = [5,4,3,2,1]/sqrt(55);   %Pulse Shape
Upsample_Factor = 5;
ts=1/Upsample_Factor;       % Time between samples (200 ms)

%% Signal Generation
%Generating Random Signal
bits = randi([0 , 1], 1,SignalSize);

%Converting the signal to PAM
PAM_Signal = (bits*2) - 1;
```

```
%Upsampling the signal
Upsampled_Signal = upsample(PAM_Signal, Upsample_Factor);

%% Convolution

%Convolving with pulse shape
Y = conv(Upsampled_Signal, p);
t_Y = 0:ts:(length(Y)-1)*ts;


%% Plot impulse train and pulse-shaped signal over time
% Time vectors (in seconds)
t_impulse = (0:length(Upsampled_Signal)-1) * ts;
figure;
subplot(3,1,1);
stem(t_impulse, Upsampled_Signal, 'filled');
title('Impulse Train (Time-Domain)');
xlabel('Time (s)');
ylabel('Amplitude');
grid on;

figure;
plot(t_Y, Y, 'r', 'LineWidth', 1.5);
title('Transmitted Signal y(t) After Pulse Shaping');
xlabel('Time (s)');
ylabel('Amplitude');
grid on;

%% Matched Filters
% A matched filter to p[n]

Matched_filter = fliplr(p);
Y_matched = conv(Y, Matched_filter);

% Time vector for matched filter output
t_matched = 0:ts:(length(Y_matched)-1)*ts;
samples= length(Matched_filter):Upsample_Factor:length(Matched_filter)+(SignalSize-
1)*Upsample_Factor;
samples_matched = Y_matched(samples);
t_samples = t_matched(samples);

% Plot matched filter output
figure;
subplot(2,1,1);
plot(t_matched, Y_matched, 'r-', 'LineWidth', 1.5);
hold on;
stem(t_samples, samples_matched, 'bo', 'filled', 'LineWidth', 1.2);
title('Output After Matched Filter ');
xlabel('Time (s)');
ylabel('Amplitude');
grid on;
legend('Matched Filter Output', 'Sampling points');
hold off;
```

```
%% Rectangular Filter
Rect_filter = [1 1 1 1 1]/sqrt(5);  % Normalized

% Filter output
Y_rect = conv(Y, Rect_filter);

% Time vector for rect filter output
t_rect = 0:ts:(length(Y_rect)-1)*ts;
samples_pos=   length(Rect_filter):Upsample_Factor:length(Rect_filter)+(SignalSize-
1)*Upsample_Factor;
samples_rect = Y_rect(samples_pos);
t_samples_rect = t_matched(samples_pos);


% Plot rect filter output
figure;
subplot(2,1,1);
plot(t_rect, Y_rect, 'r-', 'LineWidth', 1.5);
hold on;
stem(t_samples_rect, samples_rect, 'go', 'filled', 'LineWidth', 1.2);
title('Output After rect Filter ');
xlabel('Time (s)');
ylabel('Amplitude');
grid on;
legend('rect Filter Output', 'Sampling points');
hold off;




%% Correlator

correlator_output = zeros(1, length(Y));
for i = 5:5:length(Y)
    for j=0:1:4
    correlator_output(i-j) = sum(Y(i-5+1:i-j) .* p(1:end-j));
    end
end
% Pad correlator output to match length of matched filter output
corr_padded     =     [correlator_output,     zeros(1,     length(Y_matched)     -
length(correlator_output))];

% Time vector (same as matched filter for alignment)
t_corr = t_matched;
sampled_correlator = corr_padded(samples);
t_corr_samples = t_matched(samples(1:end));

% Plot Matched Filter vs. Correlator Output
figure;
subplot(2,1,1);
plot(t_matched, Y_matched, 'b', 'LineWidth', 1.1);
hold on;
```

```
stem(t_corr_samples, samples_matched, 'Ko', 'filled');
hold on;
plot(t_corr, corr_padded, 'r', 'LineWidth', 1.1);
stem(t_corr_samples, sampled_correlator, 'ko', 'filled');

legend('Matched Filter Output', 'Correlator Output (samples)','Correlator Output');
title('Matched Filter vs. Correlator Output (Continuous)');
xlabel('Time (s)');
ylabel('Amplitude');
grid on;


%% Signal Generation

% Change signal size
SignalSize = 10000;

%Generating Random Signal
bits = randi([0 , 1], SignalSize, 1);

%Converting the signal to PAM
PAM_Signal = (bits*2) - 1;
disp(PAM_Signal);

%Upsampling the signal
Upsampled_Signal = upsample(PAM_Signal, Upsample_Factor);

%Convolving with pulse shape
Y = conv(Upsampled_Signal, p);

% Time vectors for plotting
t_impulse = 0:1:length(Upsampled_Signal)-1;
t_y = 0:1:length(Y)-1;

% Plot impulse train and pulse-shaped signal
figure;
subplot(2,1,1);
stem(t_impulse, Upsampled_Signal, 'filled');
title('Impulse Train');
xlabel('Sample Index'); ylabel('Amplitude');

subplot(2,1,2);
stem(t_y, Y, 'filled');
title('Transmitted Signal y[n] (After Pulse Shaping)');
xlabel('Sample Index'); ylabel('Amplitude');


%% Filters

% A matched filter to p[n]
Matched_filter = fliplr(p);
Y_matched = conv(Y, Matched_filter);

% Time vector for matched filter output
t_matched = 0:1:length(Y_matched)-1;
```

```
% Plot matched filter output
figure;
stem(t_matched, Y_matched, 'filled');
title('Output After Matched Filter');
xlabel('Sample Index'); ylabel('Amplitude');


%Filter
Normalized_amplitude = 1/sqrt(Ts);
Rect_filter = ones(1,5) * Normalized_amplitude * 5;

Y_filtered = conv(Y, Rect_filter);

% Time vector for alternative filter output
t_alt = 0:1:length(Y_filtered)-1;

% Plot alternative filter output
figure;
stem(t_alt, Y_filtered, 'filled');
title('Output After Boxcar Filter (Normalized)');
xlabel('Sample Index'); ylabel('Amplitude');


%% Noise Analysis

% Define noise related parameters
SNR_dB = -2:1:5;  % Eb/N0 in dB from -2 dB to 5 dB
BER_matched = zeros(length(SNR_dB), 1); % To store BER for matched filter
BER_notmatched = zeros(length(SNR_dB), 1); % To store BER for rect filter
Eb_N0_linear = zeros(length(SNR_dB), 1); % Eb/N0 in linear

% Loop through each SNR value and calculate BER
for i = 1:length(SNR_dB)
    % Calculate N0 from Eb/N0
    Eb_N0 = SNR_dB(i);
    Eb_N0_linear(i) = 10^(Eb_N0/10);
    N0 = 1/(10^(Eb_N0/10));  % N0 from Eb/N0 in linear scale

    % b) Generate noise with unity variance (AWGN)
    noise = randn(size(Y));

    % c) Scale the noise to have variance = N0/2
    scaled_noise = noise * sqrt(N0/2);

    % d) Add noise to transmitted signal
    v = Y + scaled_noise;

    % Filter the noisy received signal using matched filter
    Y_matched = conv(v, Matched_filter);
    Y_notmatched = conv(v, Rect_filter);

    % BER parameters
    output_matched = zeros(1,length(SignalSize)); % Output of matched filter
    output_notmatched = zeros(1,length(SignalSize)); % Output of rect filter
```

```
    num_errors_matched = 0; % Number of errors (wrong bits) in the matched filter
output
    num_errors_notmatched = 0; % Number of errors (wrong bits) in the rect filter
output

    % Loop through each bit and calculate number of errors
    for k = 1:SignalSize
        % Hard decision with threshold at 0
        if(Y_matched(k*5) >= 0)
            output_matched(k) = 1;
        else
            output_matched(k) = -1;
        end

        % Check for error
        if(PAM_Signal(k) ~= output_matched(k))
            num_errors_matched = num_errors_matched + 1;
        end

        % Hard decision with threshold at 0
        if(Y_notmatched(k*5) >= 0)
            output_notmatched(k) = 1;
        else
            output_notmatched(k) = -1;
        end

        % Check for error
        if(PAM_Signal(k) ~= output_notmatched(k))
            num_errors_notmatched = num_errors_notmatched + 1;
        end
    end

    % Calculate BER
    BER_matched(i) = num_errors_matched / SignalSize;
    BER_notmatched(i) = num_errors_notmatched / SignalSize;
end

% Calculating theoretical BER
theoretical = 0.5 * erfc(sqrt(Eb_N0_linear));

% Plot the BER vs Eb/N0 curve
figure;
semilogy(SNR_dB, theoretical, 'k--', 'LineWidth', 1);
hold on;
semilogy(SNR_dB, BER_matched, 'b.-', 'LineWidth', 1.5);
hold on;
semilogy(SNR_dB, BER_notmatched, 'r.-', 'LineWidth', 1.5);
xlabel('Eb/N0 (dB)');
ylabel('Bit Error Rate (BER)');
legend('Theoretical',   'Matched   Filter',   'Non-Matched   Filter',   'Location',
'southwest');
title('BER (Theoretical, Matched filter, Non-matched filter)');
grid on;

%%
```

```
%EYEDIAGRAM(R, delay, CASE, Ts, Samples_Per_Symbol)
EYEDIAGRAM(0,2,1, Ts, Upsample_Factor );
EYEDIAGRAM(0,8,2, Ts, Upsample_Factor );
EYEDIAGRAM(1,2,3, Ts, Upsample_Factor );
EYEDIAGRAM(1,8,4, Ts, Upsample_Factor );

%%
function EYEDIAGRAM(R, delay, CASE, Ts, Upsample_Factor)
SRRC_filter = rcosine(1/Ts,Upsample_Factor*1/Ts ,'sqrt', R, delay);
figure;
 subplot(4,1,1);
 stem(SRRC_filter);
 title(['SRRC filter at case ', num2str(CASE)]);
 xlabel('Sample Index'); ylabel('Amplitude');
%Generating Random Signal
SignalSize_ISI = 100;
bits_ISI = randi([0 , 1], SignalSize_ISI, 1);

%Converting the signal to PAM
PAM_Signal_ISI = (bits_ISI*2) - 1;
disp(PAM_Signal_ISI);

%Upsampling the signal
Upsampled_Signal_ISI = upsample(PAM_Signal_ISI, Upsample_Factor);

%Convolving with pulse shape
Signal_Transmitted = conv(Upsampled_Signal_ISI, SRRC_filter, "same");

% Time vectors for plotting
t_impulse = 0:1:length(Upsampled_Signal_ISI)-1;
t_Signal_Transmitted = 0:1:length(Signal_Transmitted)-1;

% Plot impulse train and pulse-shaped signal
 subplot(4,1,2);
 stem(t_impulse, Upsampled_Signal_ISI, 'filled');
 title('Impulse Train');
 xlabel('Sample Index'); ylabel('Amplitude');

 subplot(4,1,3);
 stem(t_Signal_Transmitted, Signal_Transmitted, 'filled');
 title(['Transmitted Signal After SRRC filter Case ', num2str(CASE)]);
 xlabel('Sample Index'); ylabel('Amplitude');

received=conv(Signal_Transmitted, SRRC_filter, "same");
% Plot impulse train and pulse-shaped signal
t_receiver = 0:1:length(received)-1;

% Plot  filter output
 subplot(4,1,4);
 stem(t_receiver, received, 'filled');
 title(['Output After SRRC Filter Case ', num2str(CASE)]);
 xlabel('Sample Index'); ylabel('Amplitude');

eyediagram(Signal_Transmitted, Upsample_Factor);
title(['Eyediagram at A, Case ' , num2str(CASE)]);
```

```
eyediagram(received, Upsample_Factor);
title(['Eyediagram at B, Case ' ,num2str(CASE)]);
end
```